

School ID App — Master Build Document

v1.0

Purpose: This document is the high-definition first-build specification for the School ID App, intended for direct handoff into Replit so a developer can produce a strong first iteration with minimal ambiguity.

This build is for a **cross-platform mobile app** focused first on **attendance, movement, supervision, and student state awareness**. The design assumes **staff-driven scanning workflows, admin-controlled configuration**, and a **live dashboard / mission-control layer**.

1. Product Intent

The School ID App is a school operations system built around **QR code / barcode scanning** using a phone's native camera. It is designed to let staff quickly scan student IDs and update the school's real-time understanding of where students are, what status they are in, and whether any exceptions require attention.

The app is not just an attendance register. It is a **student movement and status system**.

The system should help answer, in real time:

- Who is at school
 - Who is absent
 - Who is late
 - Who has checked out early
 - Who is in class
 - Who is missing from class
 - Who is at an event or assembly
 - Who is missing from an event
 - Who is unaccounted for
-

2. Build Philosophy

This first version should be designed according to these principles:

2.1 Scan-first

The app should open into a scanning workflow, not a reporting menu. Staff should be able to act immediately.

2.2 Exception-first

Dashboards should surface problems first, not passive statistics first.

2.3 Minimal taps

After scanning a student, a staff member should usually be able to complete the action in one or two taps.

2.4 Real-time state awareness

Every scan should update the student's current state and the live dashboard.

2.5 Admin-configurable

Schools differ. Assemblies, clubs, behavior categories, detention types, events, and rule settings must be configurable by school admins.

2.6 Attendance and movement first

The first build should prioritize gate attendance, class attendance, assembly/event attendance, check-out flows, and exception logic before later-tier inventory and wallet modules.

3. Platform Assumptions

3.1 App Type

Cross-platform mobile app for Android and iPhone.

3.2 Scanner

Use native camera access for QR and barcode scanning.

3.3 Staff-Centric Use

The main operators are staff members, teachers, supervisors, office staff, event staff, and authorized student leaders in special roles.

3.4 Students

Students are mostly passive users in v1. Their ID is scanned by staff.

3.5 Admin Team

A small trusted school admin team has elevated access to backend and app configuration functions.

4. User Roles

4.1 Student

Passive participant in v1.

Their card is scanned.

Their state, attendance, behavior, and related records are linked to their profile.

4.2 Staff

Primary operational user.

Can scan students and perform authorized workflows such as:

- gate check-in
- check-out
- class attendance
- event attendance
- detention/club attendance
- merit/demerit logging

4.3 App Admin / School ID Admin

Elevated role with password-protected access to:

- bulk uploads
 - manual student/staff creation
 - data correction
 - module configuration
 - activity definitions
 - behavior category setup
 - role assignment
 - settings and thresholds
-

5. Core Functional Scope for First Iteration

5.1 Priority Modules for v1

These are the primary build targets:

A. Gate / Arrival Attendance

Staff scan students on arrival.

System records timestamp and determines on-time / late based on configured rules.

B. School Departure / Check-Out

Staff scan students out.

System records departure and reason if needed.

C. Classroom Attendance

Teachers or staff scan students into class or period attendance contexts.

System reconciles class presence with earlier gate attendance and current state.

D. Assembly / Event Attendance

Students are scanned at assemblies and events.

System tracks expected vs present and highlights missing students.

E. Activity Attendance

Detention, clubs, extracurriculars, and other activities are configurable.
Staff can start an activity and scan students into it.

F. Behavior / Discipline Logging

Staff can scan a student and assign a merit, demerit, positive credit, or transgression.
Categories are admin-defined.

G. Live Dashboard / Mission Control

Real-time operational awareness screen showing school-wide status and exceptions.

5.2 Later-Tier Modules, Not Core v1 Build Priority

These remain part of the broader vision but should not distract the first build:

A. Bookroom / Textbook Check-In and Check-Out

Link books/items to students via scan.

B. Library Circulation

Scan books and students for borrowing/return workflows.

C. Tuck Shop / Student Wallet

Load funds onto student account and deduct purchases by scan.

These should be architecturally possible later, but not the main focus of first implementation.

6. Core Student State Model

At any given moment, each student should have a current state derived from scan events and reconciliation rules.

6.1 State Values

- Not Arrived
- On Campus
- In Class

- At Event
- Checked Out
- Unaccounted

6.2 Meaning of States

Not Arrived

No valid arrival scan for the day, and no other logic placing the student on campus.

On Campus

Student has arrived at school but is not currently marked into a class or event context.

In Class

Student has been scanned into a class / lesson / period context.

At Event

Student has been scanned into assembly, event, detention, club, or another configured activity.

Checked Out

Student has been formally scanned out of school or authorized to leave.

Unaccounted

System believes the student should be somewhere in the school flow but cannot reconcile their presence correctly.

7. State Transition Logic

All major student status changes should be triggered by scan events or controlled manual overrides.

7.1 Example Transitions

- Gate scan → On Campus
- Class scan → In Class

- Event/assembly scan → At Event
- Check-out scan → Checked Out
- Completion of class/event without next confirmed location → may revert to On Campus or trigger review
- Reconciliation failure → Unaccounted

7.2 Important Principle

The app should not just store raw attendance marks.
It should maintain a **live interpreted student state**.

8. Exception Logic

The app should actively detect and surface exceptions.

8.1 Exception Examples

- Student scanned into class but no prior arrival scan
- Student expected at assembly but not present
- Student expected in class but not present
- Student scanned after being checked out
- Student marked present at school but missing from required location
- Student disappears from flow and cannot be reconciled

8.2 Dashboard Priority

Exceptions must be displayed before passive charts.

9. Navigation Structure

The staff app should use a simple bottom navigation structure.

9.1 Bottom Tabs

- Scan

- Live Dashboard
- Activities
- Students
- More / Admin

The More / Admin area can vary based on permissions.

10. Screen-by-Screen Wireframe Specification

SCREEN 1 — SCAN HOME

Purpose

Primary operational screen.

This is the most important screen in the app.

Core Idea

Open directly into live scanning mode using the device camera.

Layout

Top Bar

- School name
- current time
- logged-in staff member name
- role indicator
- optional sync / network status

Main Body

- full-width camera preview
- scanner active by default
- scanning guide frame visible

Post-Scan Bottom Sheet / Action Card

When a student is scanned, a contextual card slides up.

Student Summary Card Content

- student photo
- full name
- grade / class
- student ID number
- current state
- last seen timestamp
- optional location/context

Color-Coded State Display

- green = On Campus
- blue = In Class
- yellow = At Event
- grey = Not Arrived
- white / muted = Checked Out
- red = Unaccounted / alert state

Contextual Action Buttons

These should appear depending on role and context:

- Check In
- Check Out
- Mark Class Attendance
- Mark Event Attendance
- Register Activity Attendance
- Add Merit
- Add Demerit
- View Student Profile

UX Behavior

The screen should allow very fast repeat scanning.

After completing an action, it should return quickly to scan-ready mode.

Special Warning Banners

If the scan triggers inconsistency, show a visible warning:

- “Student not yet checked in today”
- “Student already checked out”

- “Student missing from expected class”
- “Student in conflicting state”

Data Required

- student profile
 - current state
 - latest scan event
 - rules / permissions
 - activity context if staff entered scanner from an event or class flow
-

SCREEN 2 — LIVE DASHBOARD / MISSION CONTROL

Purpose

Real-time school situational awareness.

Design Rule

This screen is exception-first, not chart-first.

Layout

Header

- title: Live Dashboard
- date
- current time
- quick refresh/live status indicator

KPI Summary Row

Cards for:

- Total Students
- Present Today
- Absent
- Late
- Checked Out
- Unaccounted

The Unaccounted card should be visually strongest when non-zero.

Critical Exceptions Panel

This sits high on the page and should be prominent.

Possible exception groups:

- Missing from Class
- Missing from Assembly
- Missing from Event
- Checked Out with Missing Reason
- Late Arrivals
- Unaccounted Students

Each item should be tappable to a filtered list.

Live Activity Feed

Reverse chronological stream of actions such as:

- John Smith checked in at 07:24
- Maria marked present in Grade 8B Science
- Thabo checked out, reason: sick
- Aiden recorded at Assembly A
- Ella received 2-point merit for leadership

Status Distribution Panel

A visual breakdown of current states:

- Not Arrived
- On Campus
- In Class
- At Event
- Checked Out
- Unaccounted

This can be cards, bars, or segmented visual blocks.

Filters

Quick filters:

- grade
- class
- event
- state
- activity type

Drill-Down Lists

Tap any KPI or exception type to see the student list behind it.

Data Required

- all scan events for the day
 - student state engine output
 - attendance summaries
 - exception engine output
 - behavior and activity updates where relevant
-

SCREEN 3 — ACTIVITIES

Purpose

Manage structured attendance outside ordinary gate and class flows.

Includes

- assemblies
- sports events
- school events
- detention
- clubs
- extracurricular attendance
- custom activity types defined by admin

Layout

Top Tabs or Filter Chips

- Assemblies
- Events
- Clubs
- Detention
- Other Activities

Activity List

Each activity card shows:

- activity name

- date/time
- responsible staff member
- expected count
- present count
- missing count
- status: upcoming / active / completed

Activity Detail Screen

When an activity is opened:

- title
- date/time
- description
- responsible staff
- expected students
- checked-in students
- missing students
- current active status

Main Actions

- Start Scanning
- Pause Scanning
- View Attendance List
- View Missing Students
- End Activity

Scan Flow

Selecting Start Scanning opens the Scan screen in activity context.
Every scan now records attendance against that activity.

Data Required

- activity definitions
- assigned students or target groups
- activity attendance logs
- live counts

SCREEN 4 — STUDENTS

Purpose

Search, inspect, and audit individual students.

Layout

Search and Filter Area

- search by name
- search by ID
- scan shortcut
- filter by grade
- filter by class
- filter by state

Student List

Each row/card shows:

- photo
- name
- grade/class
- current state
- indicator if exception exists

Student Profile Screen

Header Section

- photo
- full name
- grade/class
- student ID
- optional house / team / grouping if school uses it

Current Status Panel

- current state
- last seen time
- current or most recent location/context
- attendance summary for today

Timeline Section

A key part of the design.

Chronological timeline of the student's day:

- arrival check-in
- class scans
- event scans
- check-out
- behavior events
- activity attendance

Behavior Section

- merits
- demerits
- positive credits
- transgressions
- point totals if school uses points

Action Buttons

Depending on permission:

- Add Merit
- Add Demerit
- View Full Day Record
- Manual Override
- Force Check-In
- Force Check-Out

Data Required

- student master record
- state engine output
- scan history
- activity attendance
- behavior logs

SCREEN 5 — ADMIN / CONFIGURATION

Purpose

Administrative control of school data, modules, workflows, and settings.

Access

Restricted to authorized admin users.

Main Sections

Section A — Data Management

Functions:

- upload Excel / CSV student data
- upload staff data
- validate import fields
- view import errors
- manually add student
- manually add staff
- edit records

Important Requirement

Manual one-by-one entry must exist as fallback where bulk spreadsheet data is unavailable.

Section B — Activity Configuration

Admins should be able to create and manage activities such as:

- assemblies
- sports events
- cultural events
- detention categories
- club categories
- custom attendance contexts

Each activity type may have:

- name
- category
- expected participants
- responsible staff
- schedule/time
- attendance rules

Section C — Behavior System Configuration

Admins should be able to define:

- merit categories
- demerit categories
- positive credits
- transgressions
- labels
- point values
- whether categories are visible in reports

This is important because schools differ in discipline systems.

Section D — Roles and Permissions

Admins should be able to:

- assign staff roles
 - define who can scan for which workflows
 - define who can use admin functions
 - define who can assign behavior
 - define who can operate event attendance
-

Section E — Attendance Rules and Settings

Configurable system rules:

- school start time
 - late threshold
 - class attendance rules
 - check-out reasons
 - whether gate scan is mandatory before class scan
 - whether certain exceptions should trigger alerts
-

Section F — Module Visibility

The school should start with a default set of visible modules, but admins should be able to fine-tune them.

Default visible modules might include:

- gate attendance
- class attendance
- assembly attendance
- event attendance
- activities
- behavior

Later-tier modules can remain hidden until enabled:

- library
 - bookroom
 - tuck shop / wallet
-

11. Daily Operational Flows

11.1 Morning Arrival Flow

- staff opens Scan screen
- scans student card at gate
- system records timestamp
- system determines on-time or late
- student state becomes On Campus

If already checked in, system should warn accordingly.

11.2 Classroom Attendance Flow

- teacher opens class session or class scan mode
- scans students
- system marks them present for that class/period
- student state becomes In Class
- reconciliation checks whether student had a valid arrival scan

11.3 Assembly / Event Flow

- staff opens an assembly or event
 - enters scan mode for that event
 - scans students
 - system compares present vs expected
 - missing students surface in exception logic
-

11.4 Check-Out Flow

- authorized staff scans student
- selects checkout reason if needed
- system records timestamp and reason
- student state becomes Checked Out

Possible reasons:

- sick
 - appointment
 - parent collection
 - school-authorized departure
 - other
-

11.5 Activity Flow

- staff starts configured activity such as detention or chess club
 - scans students into that activity
 - state becomes At Event
 - attendance list is stored per activity session
-

11.6 Behavior Logging Flow

- staff scans a student
- taps Add Merit or Add Demerit
- chooses configured category

- optional note
 - save to behavior record
-

12. Dashboard Framework

The dashboard system should be tiered.

12.1 Tier 1 — Real-Time Operations Dashboard

Purpose: live situational awareness right now.

Must answer:

- who is on site
- who is absent
- who is checked out
- who is missing
- who is unaccounted

This is the most important dashboard tier for v1.

12.2 Tier 2 — Daily Management Dashboard

Purpose: same-day summary and review.

Can include:

- attendance totals
- late arrivals
- check-outs
- event attendance summaries
- behavior summaries
- unresolved exceptions

12.3 Tier 3 — Strategic / Historical Dashboard

Purpose: trends over time.

Can later include:

- weekly attendance trends
- lateness trends
- behavior trend reports
- class attendance patterns
- event participation patterns

This can be lighter in v1 and expanded later.

13. Data Model Guidance

This is conceptual and may be refined by the developer, but the first build should roughly support these core entities.

13.1 Core Tables / Collections

Students

Stores student master records.

Fields may include:

- id
- student_number
- full_name
- preferred_name
- grade
- class
- photo_url
- qr_code_value / barcode_value
- status_active

Staff

- id
- full_name
- role
- login info
- permissions

Scan Events

This is one of the most important tables.

Fields may include:

- id
- student_id
- staff_id
- event_type
- timestamp
- context_type
- context_id
- location
- metadata

Event types might include:

- check_in
- class_attendance
- event_attendance
- activity_attendance
- check_out
- behavior_log_trigger

Student Daily State

A daily computed or materialized representation of each student's current state.

Could include:

- student_id
- date
- current_state
- last_seen_at
- last_context_type
- last_context_id
- exception_flag

Activities

- id
- name
- category
- date
- start_time
- end_time
- responsible_staff_id
- expected_group_definition

- active_status

Activity Attendance

- id
- activity_id
- student_id
- scanned_at
- scanned_by_staff_id

Behavior Categories

- id
- type
- label
- point_value
- active

Behavior Logs

- id
- student_id
- category_id
- staff_id
- timestamp
- note

Check-Out Reasons

Configurable reason list.

Admin Settings

Stores school rules and thresholds.

Users / Roles / Permissions

For auth and access control.

14. API Guidance

The exact API design can vary, but the app should support endpoints roughly along these lines.

14.1 Core Endpoints

- POST /scan
- GET /students
- GET /students/:id
- GET /dashboard/live
- GET /dashboard/daily
- GET /activities
- POST /activities
- GET /activities/:id
- POST /activities/:id/attendance
- GET /behavior/categories
- POST /behavior/log
- POST /checkouts
- POST /imports/students
- POST /imports/staff
- GET /settings
- POST /settings

14.2 Realtime

Use WebSocket or Socket.IO to push updates to:

- live dashboard
 - live counts
 - activity attendance updates
 - exception changes
-

15. UX and UI Rules

15.1 General Style

The UI should feel operational, fast, clear, and low-friction.
It should not feel bloated.

15.2 Repeated Visual Rules

- state color coding should remain consistent across all screens
- buttons should be large enough for quick staff use
- error and inconsistency states must be obvious
- scanning should always feel like the main action

15.3 Mobile Practicality

This will be used in real school conditions:

- at gates
- in corridors
- in sunlight
- during busy transitions
- by staff who may be in a hurry

So screens must prioritize:

- large tap targets
 - clear status
 - rapid return to scan mode
 - minimal scrolling for common actions
-

16. Permissions Logic

Permissions should not be all-or-nothing.

Examples:

- some staff can scan gate attendance only
- some can mark class attendance
- some can run event attendance
- some can assign discipline
- only admins can import data and change rules

This matters because schools may assign limited responsibilities to different users.

17. Native Camera / Scanning Requirement

This is a critical technical requirement.

The app must be designed so that scanning works reliably on:

- Android phones
- iPhones

The developer must choose a mobile-friendly scanning implementation that uses the device camera in a way suitable for Replit-based development and deployment strategy.

This is not optional. Scanning is the operational heart of the app.

18. First-Build Prioritization Guidance

If the developer must sequence work, this is the preferred order:

Phase 1

- authentication
- roles
- student data import
- Scan screen
- gate check-in flow
- current state tracking
- basic student list/profile

Phase 2

- class attendance flow
- assembly/event attendance
- check-out flow
- live dashboard v1
- exception engine basics

Phase 3

- configurable activities
- behavior module
- admin configuration panels
- daily dashboard

- stronger reporting

Later

- bookroom
 - library
 - tuck shop / wallet
-

19. Developer Interpretation Guardrails

The developer should understand the following clearly:

19.1 This is not just an attendance register

It is a school operational state system.

19.2 The dashboard is not decorative

It is a real-time awareness tool.

19.3 Scanning is not a side feature

It is the primary operating model.

19.4 Admin configurability matters

Schools must be able to define behavior categories and activity structures.

19.5 Exceptions matter more than totals

The app must expose problems, not just record data.

20. One-Sentence Product Definition

The School ID App is a scan-first, real-time school operations app that tracks student attendance, movement, supervision status, and exceptions through QR/barcode scanning, with admin-configurable workflows and a mission-control dashboard for live situational awareness.

21. Recommended Next Step

The strongest next move is to create a **developer-facing build packet** immediately after this, containing:

- exact React screen/component structure
- recommended database schema draft
- state engine logic draft
- API contract draft
- developer build sequence checklist

I can generate that next as “**Developer Build Packet v1.0**”.